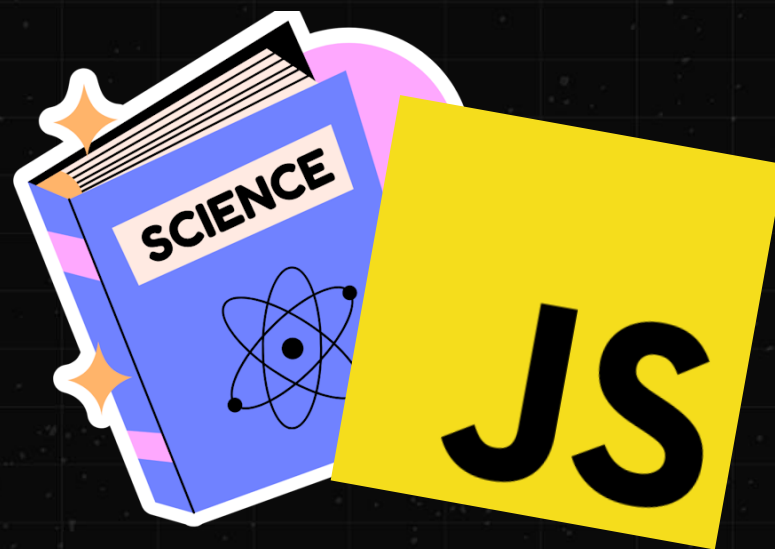


ЛЕКЦИЯ #9

CS BO FRONTEND



ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Снова говорим про массивы, но уже как **основа для других структур данных**
- * Познакомимся с важнейшими структурами данных: **вектором и матрицей**
- * Обсудим **возможные реализации и нюансы**
- * Снова поговорим про **структуры и кортежи**

А ПРИЧЕМ ТУТ
МАССИВЫ?



ВСПОМНИМ ПРО МАССИВЫ

- * **Непрерывная область памяти фиксированного размера**
логически разделенная на элементы равного фиксированного размера
- * **Константный доступ** к любому элементу по его индексу
- * **Фундаментальная структура данных** – основа для многих других

ЛИНЕЙНОЕ ПРЕДСТАВЛЕНИЕ ИНФОРМАЦИИ

- * Массив представляет **линейную последовательность элементов** одного размера
- * Элементы **располагаются друг за другом в памяти** «без пропусков»
(дырки в JS-массивах пока опустим)

А МОЖНО ЛИ
РАБОТАТЬ
С МАССИВОМ
НЕ ЛИНЕЙНО?



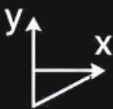
ДА

- * Вместо линейного доступа по индексу **можно создать N-мерное пространство**
- * Тогда доступ к каждому элементу будет осуществляться **с помощью указания нескольких координат**
- * Самый простой способ показать это – **двумерная или трехмерная матрицы**

2D Матрица

(строки × столбцы)

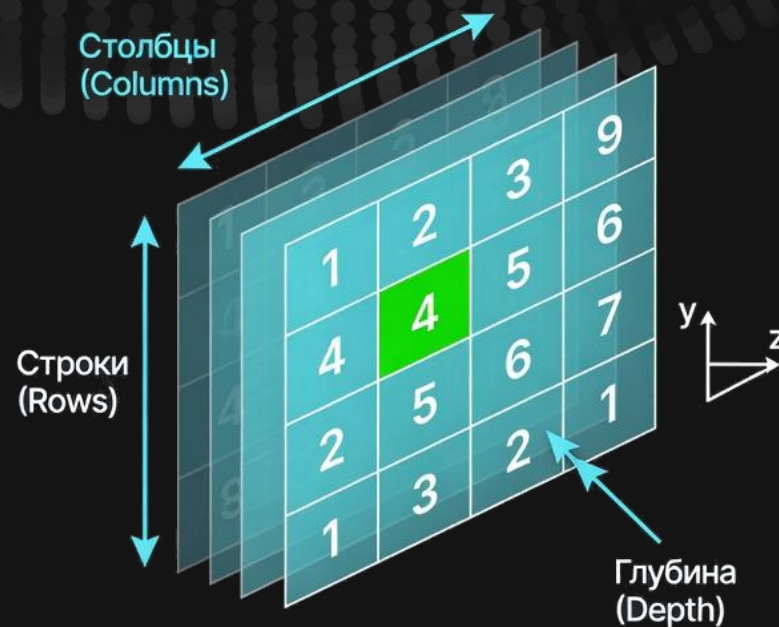
5	12	8	19	3
14	7	11	2	16
9	18	4	15	10
1	13	6	20	17



ячейка A[3,4]

3D Матрица

(глубина × строки × столбцы)



ВВЕДЕМ КОНТРАКТЫ ДВУМЕРНОЙ МАТРИЦЫ

- * Под капотом **обычный массив** или **типизированный массив**
- * Емкость массив равна **произведению строк и столбцов**
- * Чтобы вычислить индекс конкретной ячейки, **можно использовать формулу:**
$$\text{номер строки} * \text{количество столбцов в строке} + \text{номер столбца}$$
- * При таком расположении будет выгоднее **обходить матрицу по строкам**
- * Или можно использовать формулу, **оптимизированную под столбцы:**
$$\text{номер столбца} * \text{количество строк в столбце} + \text{номер строки}$$

```
class Matrix<T> {
  readonly rows: number;
  readonly cols: number;
  readonly #data: T[];

  constructor(rows: number, cols: number) {
    this.rows = rows;
    this.cols = cols;
    this.#data = new Array(rows * cols).fill(undefined);
  }

  get(row: number, col: number): T {
    return this.#data[this.#getIndex(row, col)];
  }

  set(row: number, col: number, value: T) {
    this.#data[this.#getIndex(row, col)] = value;
  }

  #getIndex(row: number, col: number) {
    return row * this.cols + col;
  }
}
```

```

enum MatrixOrder { RowMajor, ColumnMajor, DepthMajor }

class Matrix3D<T> {
  readonly rows: number;
  readonly cols: number;
  readonly depths: number;
  readonly #data: T[];
  readonly #order: MatrixOrder;

  constructor(rows: number, cols: number, depths: number, order = MatrixOrder.RowMajor) {
    this.rows = rows;
    this.cols = cols;
    this.depths = depths;
    this.#order = order;
    this.#data = new Array(rows * cols * depths).fill(undefined);
  }

  get(row: number, col: number, depth: number): T {
    return this.#data[this.#getIndex(row, col, depth)];
  }

  set(row: number, col: number, depth: number, value: T) {
    this.#data[this.#getIndex(row, col, depth)] = value;
  }

  #getIndex(row: number, col: number, depth: number) {
    switch (this.#order) {
      case MatrixOrder.ColumnMajor: return col * this.rows * this.depths + row * this.depths + depth;
      case MatrixOrder.DepthMajor: return depth * this.rows * this.cols + row * this.cols + col;
      default: return row * this.cols * this.depths + col * this.depths + depth;
    }
  }
}

```


РЕФЛЕКСИРУЕМ

- * Вся информация **хранится последовательно в памяти**
- * Но у нас появляется возможность обращаться к ячейкам **по составному индексу**
- * Это очень удобно для представления различной информации, особенно **где используется линейная алгебра и операции над матрицами**
- * Например, графики, координат или систем уравнений
- * Легко **оптимизировать хранения данных в памяти** под наилучшее использование кэш-памяти

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВСЕ НЕ ТАК СЛОЖНО

- * Матрица – это пример такой структуры, которая **в памяти хранится абсолютно также, как простой массив**
- * Но вводит особые контракты на обращение к элементам
- * Главный плюс матриц – это удобство представления такого рода информации

А ЧТО С
ЭФФЕКТИВНОСТЬЮ?



ВСЕ ХОРОШО

- * Массивы (особенно типизированные) хорошо работают с кэш-памятью
- * А тут еще **связанная информация (оси x , y , z и т.д.) хранится близко в памяти**
- * Поэтому такое представление используется для решения самых разных задач
- * Разумеется, можно масштабировать матрицу, **чтобы хранить N -мерное представление**

ЭТО **ВМЕСТО**
МАССИВА
МАССИВОВ?



КАК ВАРИАНТ

- * Преимущество массива массивов в том, что его элементы **могут иметь разную длину**
- * Если вам это условие не важно, то, зачастую, **эффективнее использовать матрицу**
- * В случае с обычными массивами вы не только эффективнее используете кэш, но и **уменьшаете работу сборщику мусора** (не надо удалять подмассивы)

НО ЭТО НЕ ВСЕ ПЛЮСЫ

- * Сериализация массива массивов потребует кодировать дополнительную информацию для декодирования
- * Сложность сериализации и десериализации линейно зависит от данных
- * А в случае с матрицей можно сделать обычный view над данными
- * При использовании типизированных массивов сложность будет константа

А ЕСЛИ МАССИВ
КОРТЕЖЕЙ
ИЛИ СТРУКТУР?



ЭТО ЕЩЕ ОДНО РЕШЕНИЕ

- * Вместо (или вместе) создания нескольких осей и координат просто **храним элементы в виде структур или кортежей**
- * Появляется возможность **хранить рядом элементы разных типов**
- * **Для обычных массивов** это выглядит как массив массивов или массив объектов
- * А вот для типизированных массивов мы можем **закодировать значение внутри одного элемента массива**

```

class Tuple {
  static readonly BYTES_PER_ELEMENT = 8;

  static getFirst(view: DataView, offset = 0, littleEndian = true): number {
    return view.getInt32(offset, littleEndian);
  }

  static setFirst(view: DataView, offset = 0, value: number, littleEndian = true) {
    view.setInt32(offset, value, littleEndian);
  }

  static getSecond(view: DataView, offset = 0, littleEndian = true): number {
    return view.getFloat32(offset + 4, littleEndian);
  }

  static setSecond(view: DataView, offset = 0, value: number, littleEndian = true) {
    view.setFloat32(offset + 4, value, littleEndian);
  }
}

```

```

class TupleArray {
  readonly buffer: ArrayBuffer;
  readonly BYTES_PER_ELEMENT = Tuple.BYTES_PER_ELEMENT;
  readonly #view: DataView;

  constructor(length: number) {
    this.buffer = new ArrayBuffer(length * this.BYTES_PER_ELEMENT);
    this.#view = new DataView(this.buffer)
  }

  get(index: number): [number, number] {
    const i = this.#getIndex(index);
    return [Tuple.getFirst(this.#view, i), Tuple.getSecond(this.#view, i)];
  }

  set(index: number, value: [number, number]) {
    const i = this.#getIndex(index);
    Tuple.setFirst(this.#view, i, value[0]);
    Tuple.setSecond(this.#view, i, value[1]);
  }

  getFirst(index: number): number {
    return Tuple.getFirst(this.#view, this.#getIndex(index));
  }

  setFirst(index: number, value: number): void {
    Tuple.setFirst(this.#view, this.#getIndex(index), value);
  }

  getSecond(index: number): number {
    return Tuple.getSecond(this.#view, this.#getIndex(index));
  }

  setSecond(index: number, value: number): void {
    Tuple.setSecond(this.#view, this.#getIndex(index), value);
  }

  #getIndex(index: number) {
    return index * this.BYTES_PER_ELEMENT;
  }
}

```

```

const tuples = new TupleArray(5);

tuples.set(0, [42, 3.14]);
tuples.set(1, [100, 2.718]);

console.log(tuples.get(1)); // [100, 2.718]
console.log(tuples.getSecond(1)); // 2.718

```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

БЕЗ ПАНИКИ, МЫ УЖЕ ТАКОЕ ДЕЛАЛИ

- * В уроке по созданию структур поверх типизированных массивов
- * В одной из ДЗ вы уже кодировали кортежи для строк и их длин
- * Принцип тут не меняется – используя типизированные массивы или [DataView](#) вводим контракты на работу с элементами
- * Можно обобщить реализацию и создать стандартное решение
- * Или использовать стороннее, например, [flatbuffers](#)

В ЧЕМ ПРОФИТ

- * Если использовать типизированные массивы и такой подход, то мы получаем **константное время сериализации и парсинга**
- * **Отличную работу с кэш-памятью** и общее низкое потребление памяти
- * **Оптимизация работы сборщика мусора** – изменения внутри типизированного массива его не волнуют

КОГДА, ЧТО
ИСПОЛЬЗОВАТЬ?



ДАВАЙТЕ РАЗБИРАТЬСЯ

- * Структура или кортеж используются, чтобы объединить несколько значений (не обязательно одного типа) **в рамках одного целого**
- * Массив или матрица – это про **создание списка или N-мерного представления** элементов одного размера и типа
- * Для представления списка пикселей правильнее сделать кортеж/структуру RGBA и сделать матрицу таких кортежей
- * Потому что пиксель – это хоть и 4-ре разных значения, но **они представляют целое**

С ОБЫЧНЫМИ МАССИВАМИ

- * Нужно держать в голове косвенность (массив значений vs массив указателей)
- * Но можно выделить простое правило: **если элементы получаются разного типа**, то лучше использовать массив массивов или массив объектов
- * А **если элементы одного типа** (например, числа или строки), то лучше использовать плоское представление

А КАК ДЕЛАЮТ МАССИВЫ РАСШИРЯЕМЫМИ?



НЕЛЬЗЯ РАСШИРИТЬ СОЗДАННЫЙ МАССИВ

- * Но можно создать **новый массив с подходящей емкостью**
- * Например, в два раза больше предыдущего
- * И **скопировать туда все элементы из исходного**, а старый удалить
- * Чтобы организовать такую логику нам необходимо хранить **емкость массива и его длину, а также ссылку на «сырой» массив**
- * Поэтому **организуется структура из 3-х значений**

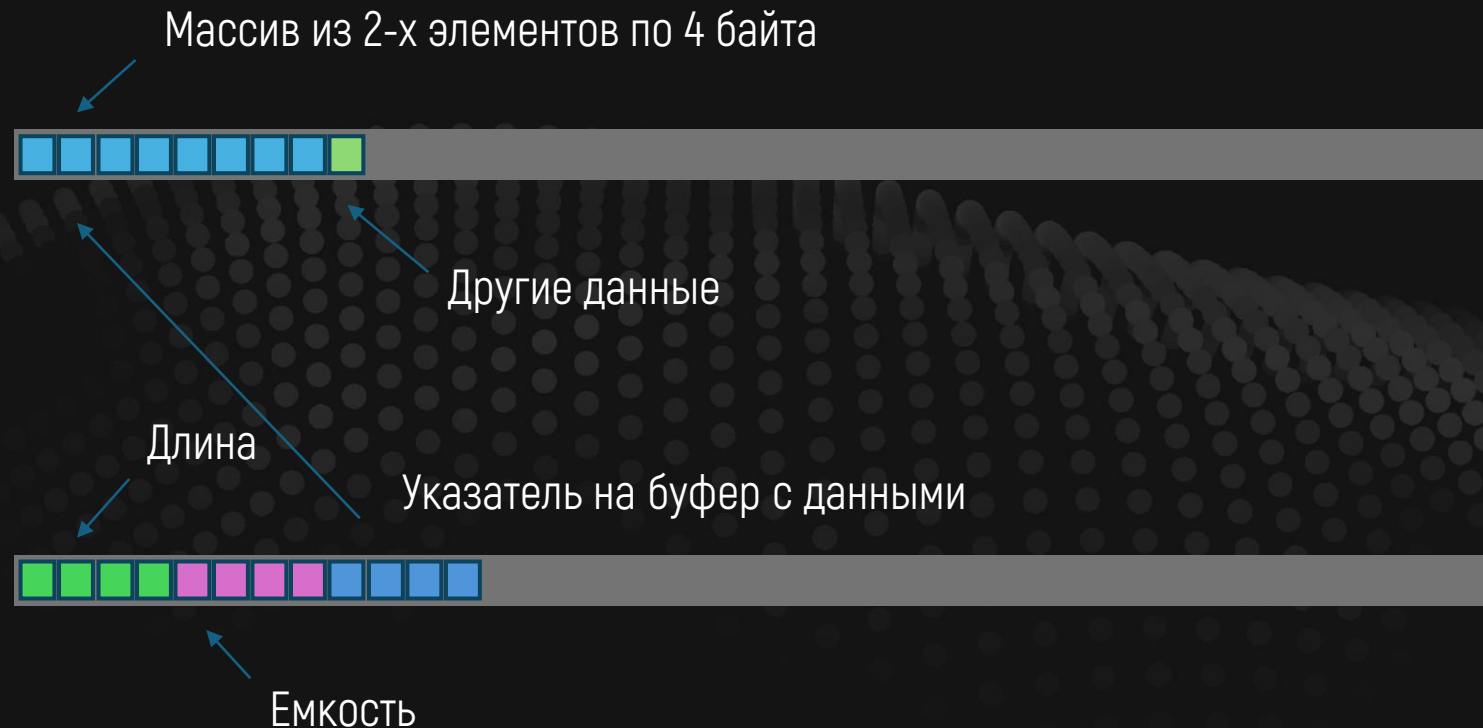
ЗНАКОМЬТЕСЬ, ВЕКТОР

```
struct Vector {  
    capacity: uint;  
    length: uint;  
    buffer: &ArrayBuffer;  
}
```

ПРИНЦИП ВЕКТОРА ОЧЕНЬ ПРОСТ

- * Аллоцируем массив элементов заданной емкости
- * Когда добавлять новые элементы становится невозможно (емкость равна длине) – аллоцируем новый массив большей емкости
- * Копируем все элементы исходного
- * И заменяем ссылку в структуре на новый

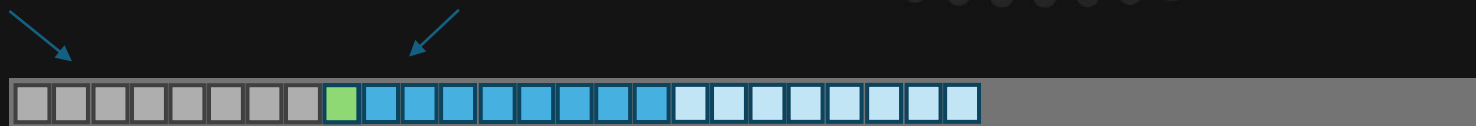
ПОЛНОСТЬЮ ЗАПОЛНЕННЫЙ ВЕКТОР



АЛЛОЦИРУЕМ НОВЫЙ БУФЕР

Старый буфер освобождаем

Элементы старого массива были скопированы



Длина

Поменяли указатель на буфер



Емкость



А НЕЛЬЗЯ ЛИ
НЕ КОПИРОВАТЬ
ПАМЯТЬ ПРИ
РАСШИРЕНИИ?



МОЖНО В НЕКОТОРЫХ СЛУЧАЯХ

- * Если в «окрестности» буфера достаточно места для расширения, то можно обойтись без избыточного копирования памяти (REALLOC)
- * На практике так часто и делается, но такая оптимизация не всегда возможна



```
// Закладываем возможность роста  
const buf = new ArrayBuffer(2, {maxLength: 16});  
buf.resize(8); // VM потенциально может оптимизировать расширение
```

А МОЖНО
СРАЗУ ЗАДАТЬ
ПРАВИЛЬНУЮ
ЕМКОСТЬ?



НУЖНО

- ✱ Если необходимая или минимальная емкость известна заранее, то **нужно сразу аллоцировать буфер нужного размера**
- ✱ Минимизация лишних аллокаций – одна из важнейших оптимизаций

НАПРИМЕР, РАЗНИЦА В 4-РЕ РАЗА!

```
console.time("push");

const a = [];

for (let i = 0; i < 1e7; i++) {
  a.push(i);
}

console.timeEnd("push");
```

```
console.time("fixed capacity");

const b = new Array(1e7);

for (let i = 0; i < 1e7; i++) {
  b[i] = i;
}

console.timeEnd("fixed capacity");
```

ВЕКТОР КАК ФУНДАМЕНТАЛЬНАЯ СТРУКТУРА

- * Вектора являются сердцем многих других структур данных
- * Например, изменяемые строки (строковые буферы) или хэш-таблицы
- * А значит и в этих структурах данных предварительное задание емкости и борьба с избыточным копированием памяти крайне важна
- * К сожалению, в JS мы не всегда можем этим управлять

ПОЛУЧАЕТСЯ,
ЧТО ОБЫЧНЫЕ
МАССИВ В JS –
ЭТО ВЕКТОРА?



И ДА, И НЕТ

- * **Спецификация не говорит**, какая конкретно структура данных должна реализовывать массивы
- * Более того, **массивы в JS могут быть разреженными или ассоциативными** – в таких случаях внутренняя реализация еще усложняется
- * Но в обычном случае там вектор, как и в других ЯП

НАПРИМЕР, В V8

- * Packed Elements – массив без «дырок», все элементы последовательны, хранится как непрерывный вектор
- * Holey Elements – режим для разреженных массивов, движок все еще использует вектор, но помечает «дырки» специальным значением
- * Dictionary Elements – самый медленный режим, используется, когда массив становится *очень* разреженным (вместо вектора создается хэш-таблица)

```
const arr = [];  
arr[50000] = 1;  
%DebugPrint(arr); // DICTIONARY_ELEMENTS
```

А ЧТО ПРОИСХОДИТ
ПРИ УМЕНЬШЕНИИ?



ПО РАЗНОМУ

- * Обычно, при удалении элементов из вектора **буфер не меняется**
- * Но, **может быть явный метод для «ужатия»** внутреннего буфера
- * В JS спецификация никак это не регламентирует, но мы видели на бенчмарках в V8, что **для Packed массивов такое происходит**

ЕСЛИ В JS УЖЕ
ЕСТЬ ВЕКТОР,
ТО ЗАЧЕМ НАМ
ЗНАТЬ ПРО ЭТО?



КОНКРЕТНЫЕ ПРИЧИНЫ

- * Чтобы не писать код, который будет тормозить из-за реаллокаций
- * Реализация вектора для типизированных массивов
- * Запомнить паттерн амортизированного роста – его можно использовать далеко за пределами векторов
- * Например, при реализации загрузки данных для бесконечного скrolла

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

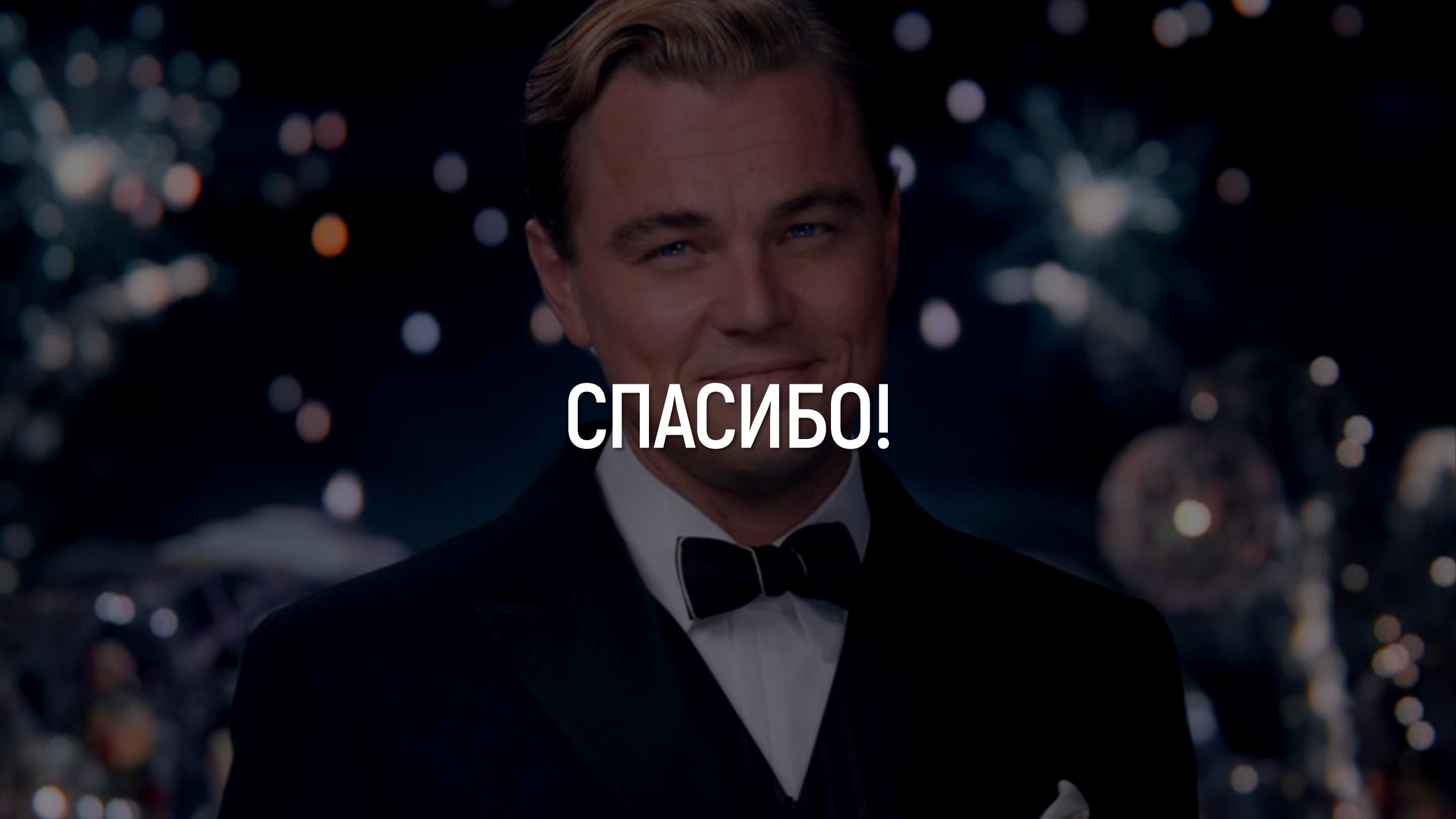
СТОЯНА, СТОЯНААА!

ЧТО ВАЖНО ПОНЯТЬ

- ✱ **Расширяемость массивов** – это не «магия»
- ✱ Она **построена на фундаментальных структурах данных:**
массив, структура и указатель
- ✱ И отдельных контрактах на добавление и удаление

ПОДВЕДЕМ ИТОГИ

- * Мы увидели, как **одни структуры данных строятся из других**
- * Например, **матрица – это просто контракт над массивом**
- * А вектор строится на **использовании массива и структуры над ним** со специальными контрактами
- * Хотя в JS нет структур и кортежей, но мы можем сделать их используя типизированные массивы и **DataView**

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with numerous out-of-focus, colorful bokeh lights in shades of blue, orange, and white, suggesting a night-time event or fireworks.

СПАСИБО!